

CIS 4020 · DATA STRUCTURES

Algorithm Analysis

Counting steps, growth rates, and Big-O, worked all the way through

A word before you read. These are my notes from years of teaching this unit, cleaned up so they are readable. For these notes, I have been using AI to help type them up, check for issues, and fill in explanations, which means a wrong or imprecise statement can slip in and survive longer than it should, and I do not fully trust the AI, so I am still editing. They are **not** meant to be a study guide, they are not gospel, and not everything here shows up in a lecture (and some lecture items are not here). If you find something that looks wrong, contradicts the book, or just does not behave the way I claim, **tell me**. You will be doing your classmates and me a favor, and you will understand it better for having caught it. The textbook is the tiebreaker. Unlike most units this semester, the ideas here are identical in Java and C/C++, so there is no side-by-side code split this time.

1. Why we do not just time it

The obvious way to compare two algorithms is to run both and see which one is faster. This does not work as well as it sounds. A stopwatch measures one run, on one machine, under one set of conditions, on one day. Run the exact same program four times and you can get four different nanosecond readings, none of which tell you what happens when the input gets ten times bigger, or what happens on a different computer entirely.

What we want instead is a way to describe how the amount of work an algorithm does grows as its input grows, independent of hardware, compiler, or which run happened to catch a lucky moment. That is the whole point of algorithm analysis: count the operations an algorithm performs, as a function of the input size (almost always called n), instead of counting seconds.

2. Counting steps: three ways to solve FizzBuzz

The running example for this unit: print every integer from 1 to 100. For multiples of 3, print “fizz”; for multiples of 5, print “buzz”; for multiples of 15, print “fizzbuzz.” Three algorithms below all solve this correctly, and all three print exactly the same output. What differs is how much work each one does.

Algorithm A

```
if (i % 15 == 0)
    print("fizzbuzz")
if (i % 3 == 0 &&
    i % 15 != 0)
    print("fizz")
if (i % 5 == 0 &&
    i % 15 != 0)
    print("buzz")
```

Algorithm B

```
if (i % 15 == 0)
    print("fizzbuzz")
else if (i % 3 == 0)
    print("fizz")
else if (i % 5 == 0)
    print("buzz")
```

Algorithm C

```
if (i % 3 == 0)
    print("fizz")
if (i % 5 == 0)
    print("buzz")
```

Algorithm A uses three independent ifs, each guarded by a check that the number was not already caught by the multiple-of-15 case. Algorithm B uses a single if/else-if/else-if chain, so only one

branch ever runs. Algorithm C uses two completely independent ifs with no guards at all; for a multiple of 15, both fire, printing “fizz” and then “buzz” back to back, which is how “fizzbuzz” appears without a dedicated case for it.

Algorithm A: a rough estimate

Each integer can be checked against up to five conditions (the 15-check, then up to two comparisons for the 3-branch, then up to two for the 5-branch). As a rough estimate: $100 \times 5 = 500$ total checks. This number is deliberately loose. The real count depends on the order the conditions are written in, the order of the “and”s, whether “and” short-circuits, and the calls to print themselves.

Algorithm B: an exact count

Because only one branch runs per integer, we can count exactly.

Table 1: Exact check count for Algorithm B

Group	Count of integers	Checks each	Subtotal
Multiples of 15	6	$\times 1$	6
Multiples of 3 (not 15)	27	$\times 2$	54
Multiples of 5 (not 15)	14	$\times 3$	42
Everything else	53	$\times 3$	159

Total: $6 + 54 + 42 + 159 = 261$ checks.

Algorithm C: an exact count

No guards, no chaining: every integer is checked exactly twice, once against 3 and once against 5, regardless of outcome. $100 \times 2 = 200$ checks total.

So which one wins?

Rough numbers: A ≈ 500 , B is 261, C is 200. Algorithm C looks best by this measure. But before trusting that ranking, we should ask what we are actually counting.

3. What exactly counts as one step?

Look at a single line from Algorithm B:

```
if ((i % 15) == 0)
    print("fizzbuzz")
```

This “one line” is really at least three operations: a **modulus** (`i % 15`), a **comparison** (`== 0`), and a **print**. Every branch in Algorithm B contains at least one modulus and one comparison, and some contain a print as well. If we count lines, we undercount. If we count comparisons only, we ignore the modulus and the print, which are not free either. From here on, we analyze algorithms by their execution steps, not by lines of code or by a single kind of operation.

4. A harder example: the multiplication table

The FizzBuzz example only has one loop. Here is a problem with two nested loops: print a formatted multiplication table from a minimum to a maximum value. For example, from -3 to 1 :

Table 2: Multiplication table from -3 to 1

$\times$	-3	-2	-1	0	1
-3	9	6	3	0	-3
-2	6	4	2	0	-2
-1	3	2	1	0	-1
0	0	0	0	0	0
1	-3	-2	-1	0	1

Here is the Java that builds it, with $n = \max - \min + 1$ standing for the number of rows (and columns):

```
1 int result = 0;
2 for (int j = min; j <= max; j++) {
3     System.out.print(j + " ");
4     for (int i = min; i <= max; i++) {
5         result = j * i;
6         System.out.print(result + " ");
7     }
8     System.out.println();
9 }
```

Counting a for-loop honestly

A **for** loop has three parts, and they do not all run the same number of times. For a loop that runs n times, the initializer runs once, the test runs $n + 1$ times (once per iteration, plus one final check that fails), and the increment runs n times. Line 2, the outer loop, costs $1 + (n + 1) + n = 2n + 2$ for the loop mechanics alone, and that cost is paid once, for the loop as a whole, not once per iteration. Line 4, the inner loop, follows the identical pattern, except it restarts fresh on every one of the n outer iterations.

Table 3: Line-by-line step count

Line(s)	Count
Line 1	1
Line 2 (outer for, whole loop)	$1 + (n + 1) + n = 2n + 2$
Line 3	n
Lines 4–6 (inner for, once per outer iteration)	$(1 + (n + 1) + n + n + n) \times n = (4n + 2)n = 4n^2 + 2n$
Line 8	n

Adding it all up:

$$1 + (2n + 2) + n + (4n^2 + 2n) + n = 4n^2 + 6n + 3.$$

Size matters: the step count depends on n , the “problem size,” not on some fixed number of operations.

A thought experiment

Plugging small values of n into $4n^2 + 6n + 3$:

Table 4: Steps as n grows

n	1	2	3	4	5	6
steps	13	31	57	91	133	183

Which term becomes the greatest factor as n grows? What does that term represent in the algorithm? Those two questions are the entire point of the next section.

5. From loops to growth rates

A single loop over n items costs $O(n)$. Two nested loops over n items cost $O(n^2)$, which is exactly the pattern above. Three nested loops cost $O(n^3)$. What matters most is the fastest-growing term, not the exact formula behind it.

Table 5: Common growth-rate classes

Notation	Name
$O(1)$	constant
$O(\log_2 n)$	logarithmic
$O(n)$	linear
$O(n \log_2 n)$	loglinear (linearithmic)
$O(n^2)$	quadratic
$O(n^3)$	cubic
$O(2^n)$	exponential

Why the fastest-growing term is the only one that matters

Table 6: Problem size vs. estimated steps

Function	$n=10$	100	1,000	10,000	100,000	1,000,000
$O(1)$	1	1	1	1	1	1
$O(\log_2 n)$	3	6	9	13	16	19
$O(n)$	10	10^2	10^3	10^4	10^5	10^6
$O(n \log_2 n)$	30	664	9,965	10^5	10^6	10^7
$O(n^2)$	10^2	10^4	10^6	10^8	10^{10}	10^{12}
$O(n^3)$	10^3	10^6	10^9	10^{12}	10^{15}	10^{18}
$O(2^n)$	10^3	10^{30}	10^{301}	$10^{3,010}$	$10^{30,103}$	$10^{301,030}$

Doubling the problem size barely moves an $O(1)$ or $O(\log n)$ algorithm, but it is catastrophic for an $O(2^n)$ one. This table is the entire reason growth rate matters more than any single measurement.

6. Rules of Big-O

- **Drop lower-order terms.** They stop mattering once n is large.
- **Drop constant multiplicative factors.** They do not change how fast something grows.
- **When unsure, usually estimate up.**
- **You can add growth rates:** $O(f(n)) + O(g(n)) = O(f(n) + g(n))$.

Applying these to our own formula: $4n^2 + 6n + 3$. Drop the lower-order terms ($6n$ and 3): $4n^2$. Drop the constant factor (4): n^2 . So the multiplication table program is $O(n^2)$, exactly matching the “nested loop” intuition from the previous section.

7. Best, worst, and average case

The same algorithm, on the same size input, can do very different amounts of work depending on what that input actually looks like. Imagine a sorting algorithm.

- **Best case.** The data is already sorted. This does not happen often (Murphy’s Law), and we tend to ignore it, but we should not dismiss it outright.
- **Average case.** The items are uniformly, randomly distributed. This is the more typical situation in practice.
- **Worst case.** The items start in reverse order. This may not be common, but Murphy’s Law says it will happen eventually, which is why the worst case is often the best judge of an algorithm.

These scenarios are not the same for every algorithm; what counts as “worst” for one sorting algorithm might not be the worst arrangement for a different one.

Sequential search

Begin with the first item, look at each one in turn, and stop as soon as you find it (or reach the end).

Table 7: Sequential search by case

Case	Big-O	Why
Worst	$O(n)$	It’s the last item, or not present at all
Average	$O(n)$	Somewhere in the middle; really less than a full n , but still grows with n
Best	$O(1)$	It’s the first item

The best case only describes one special situation. We focus on the worst and average cases.

Binary search: cutting the problem in half

Binary search repeatedly cuts the remaining input in half: $n, n/2, n/4, \dots$, down to a single item, after k divisions.

If the number of items is a power of two, then $n = 2^k$. Solving for k : $k = \log_2 n$. So binary search takes $O(\log_2 n)$ steps in the worst case.

If the number of items is not a power of two, then n sits strictly between two consecutive powers of two: $2^{k-1} < n < 2^k$. Taking $\log_2$ of all three parts (a valid step, since $\log_2$ is increasing):

$$k - 1 < \log_2 n < k.$$

So k is always within 1 of $\log_2 n$, whichever way you round. That extra 1 is a constant, and constants drop out of Big-O, so either way the result is $O(\log_2 n)$.

Splitting the input in half, again and again, often leads to $O(\log_2 n)$ or $O(n \log_2 n)$.

8. Common pitfalls

- **Treating a single run's count as the growth rate.** Counting exactly how many checks Algorithm B does for $n = 100$ tells you one data point, not how it scales. Big-O describes the shape of growth, not a single number.
- **Forgetting that Big-O is an upper bound.** An algorithm that is truly $O(1)$ is technically also $O(n)$, since $O(n)$ just means “grows no faster than n .” In practice we always state the tightest bound we can prove, but the notation itself does not forbid the looser one.
- **Assuming the worst case is the only one that matters.** The average case is often what you actually observe in practice; report whichever case the situation calls for, and say which one you mean.
- **Confusing a faster measured run with a better growth rate.** An $O(n^2)$ algorithm can easily outrun an $O(n \log n)$ algorithm on a small input, because Big-O only describes what happens as n gets large, and it says nothing about the constant factors that dominate at small sizes.
- **Forgetting that constants can matter in practice, even though Big-O ignores them.** Dropping the “4” in $4n^2$ is correct for classification, but it does not mean that constant has zero real-world cost.

9. Where this lives in the textbooks

- **Java, 3rd edition (Carrano).** Chapter 4, The Efficiency of Algorithms (pp. 87–104), covers this entire unit: measuring efficiency, counting basic operations, best/worst/average case, Big-Oh notation, the complexities of common program constructs, and picturing efficiency, immediately followed by an efficiency comparison of two implementations of the ADT Bag.
- **C/C++, 8th edition (Carrano and Henry).** Chapter 7, Algorithm Efficiency (7.1 What Is a Good Solution?, 7.2 Measuring the Efficiency of Algorithms), sitting right after the recursion chapters and right before the stacks chapters. Sorting, where this vocabulary gets its first big workout, follows in Chapters 14 and 15.